

Computer Organization & Architecture – BE04000251

Unit- 8 Input – Output Organization

**Prepared By: Sweetu D.
Sureja**

**Information Technology Department
V.V.P. Engineering College, Rajkot.**

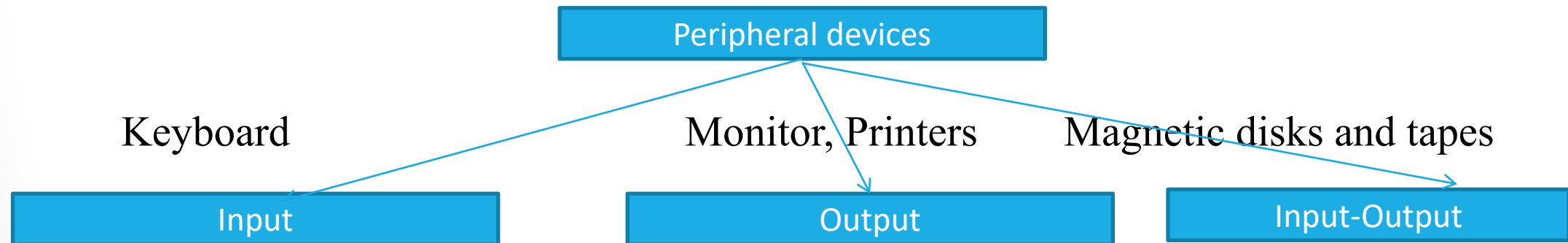
Terminology

What is I/O ?

users are from world that directly not interact with system, so I/O device provides a path to user given input and taken output.

What is Peripheral devices?

I/O devices attached to the Computers are called peripheral devices.



Input – Output Interface

- I/O devices and CPU can not communicate directly, we need special communication link between them, that's why both are easily communicated. it is called I/O interface.

Why Processor(CPU)and I/O directly not Communicated ?

Differences between Computer (CPU)and peripheral devices(I/O):

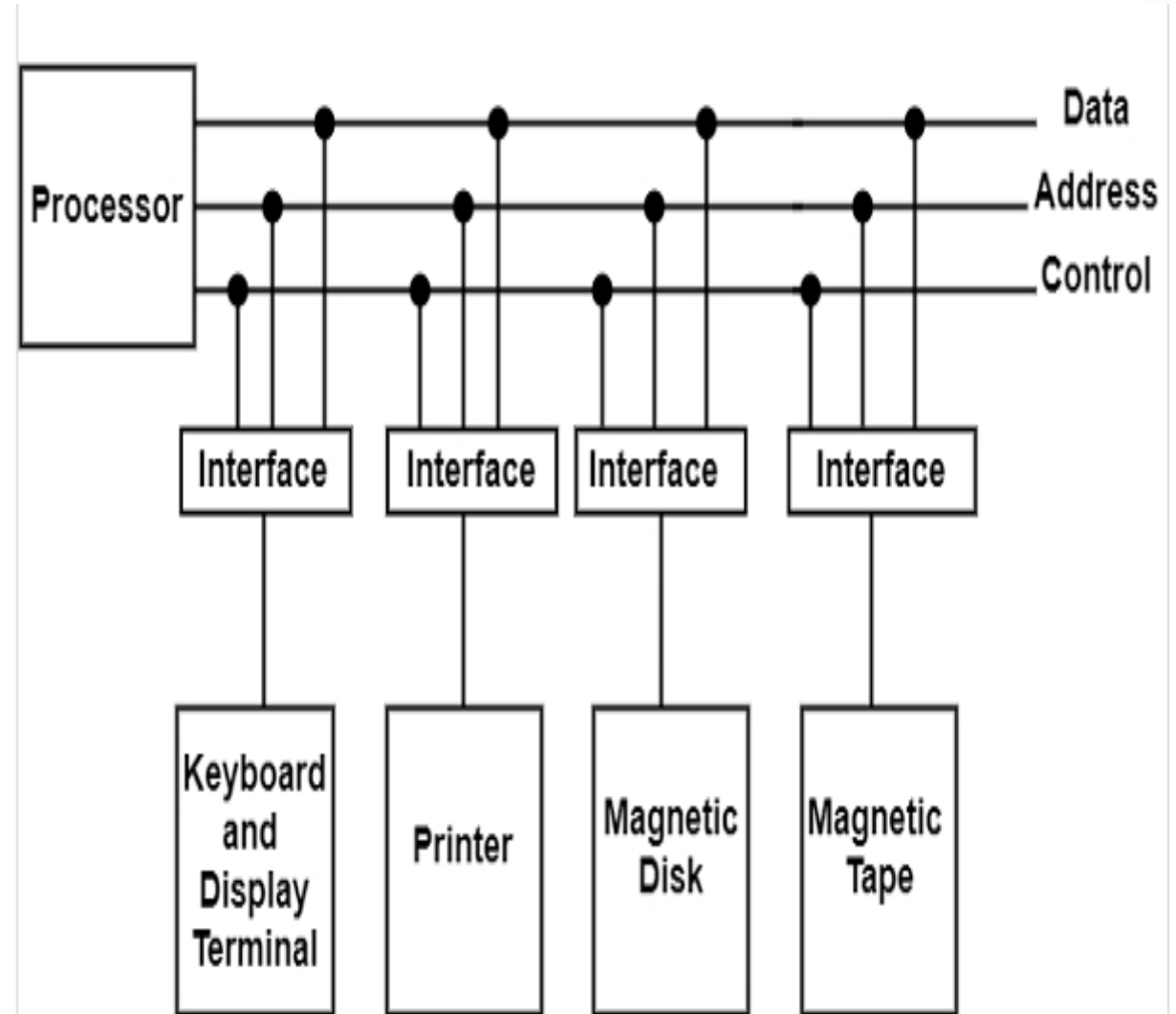
- Data transfer Rate(CPU has faster than Peripherals)
- Unit of Information (Instruction code format are different)I/O – Bytes/Blocks and CPU – word format
- To resolve these differences we use interface units that act as a interface between CPU and peripheral device.

Input – Output Interface

What is work of I/O interface?

Provides method for transferring information between internal storage (Processor) and external I/O devices.

- The I/O bus includes data lines, address lines, and control lines.
- Here, in fig Processor directly not connected to the I/O devices,
- It is connected through the interface unit.



Input – Output Interface

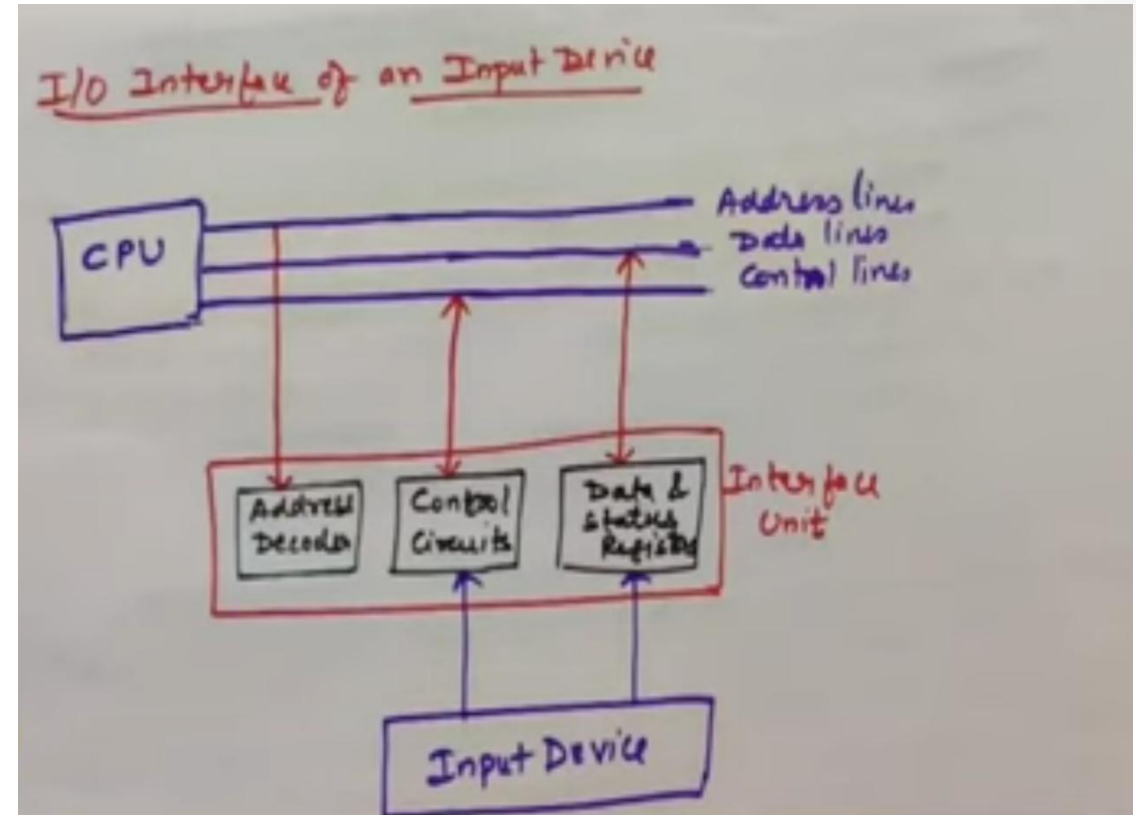
Interface unit

- 1) Address decoder
- 2) Control Circuits
- 3) Data and Status Registers

Address Decoder – it is connected with address line, Enables the device to recognize its address when the address appears on the address lines.
If it's own interface address then accept it and process it.

Data Register – I/O devices data passes to the data lines which also stored and hold the data, also it receives data From data line.

Status Register – it store the status information.



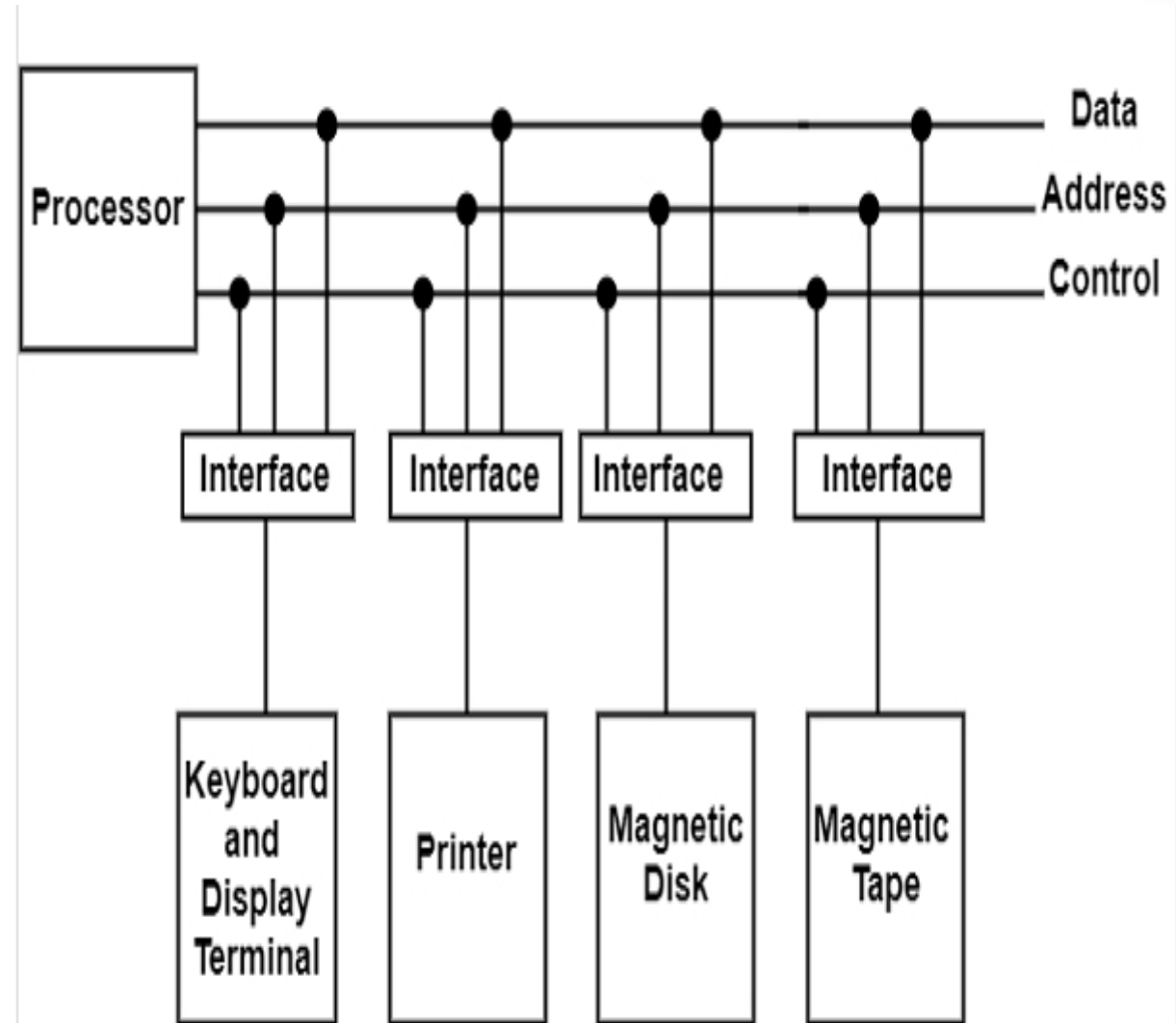
Control Circuits – it **synchronizes data flow** and **supervise** the data transfer between processor and peripheral

Input – Output Interface (Workflow)

- Processor **Pass the address** through the address line then
- Each **interface Unit**,
 - ✓ **Decodes the device address**, it only match address accept it, other line deactivated (Discard) it.
 - ✓ Suppose, selected Keyboard and display terminal

Processor send **function code** (I/O command) through control lines.

- ✓ Decodes the commands(operation)
- ✓ Synchronizes data flow and supervises the transfer between peripheral and Processor



Input – Output Interface

- It is send by Processor, it is instructions that executed in interface and attached Peripheral devices.

- I/O commands have four types :-

1. Control Command:

It activate the peripheral devices and inform what to do.

2. Status Command:

It used to test various status like busy, free or error condition in peripheral and an interface (Check the status of peripheral device before transfer is initiated.)

3. Data output Command:

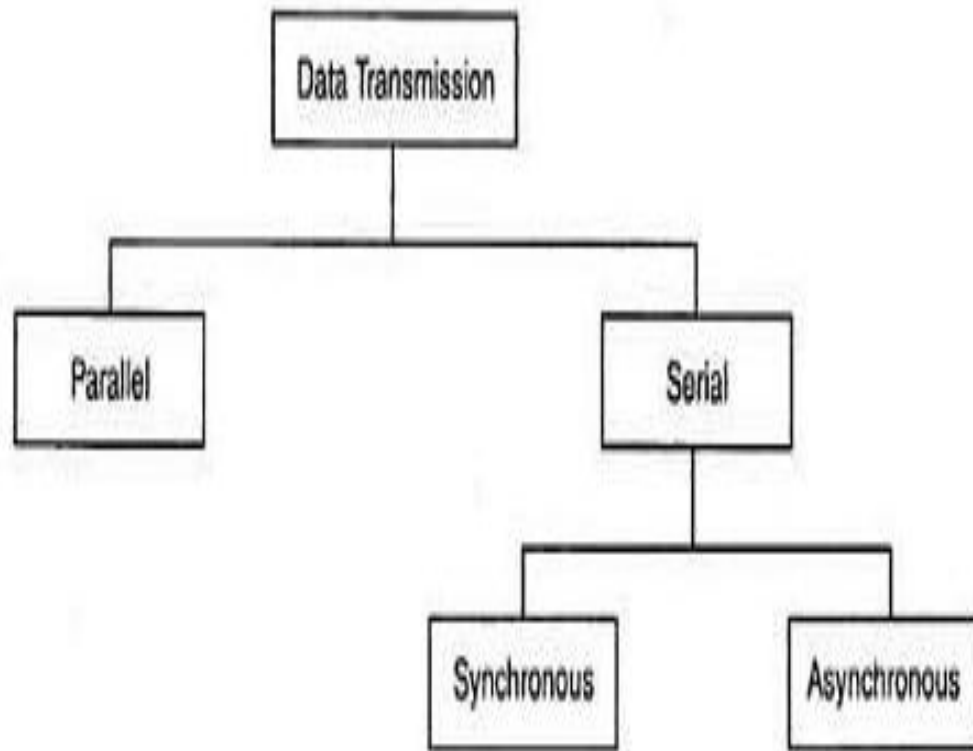
It transferring the data from bus into its own interface register. When peripheral is free then it will collect it (I/O write)

4. Data input Command:

It transferring the data from Peripheral to interface stored it when Processor free then it will read it..(I/O read)

Data Transmission (Terminology)

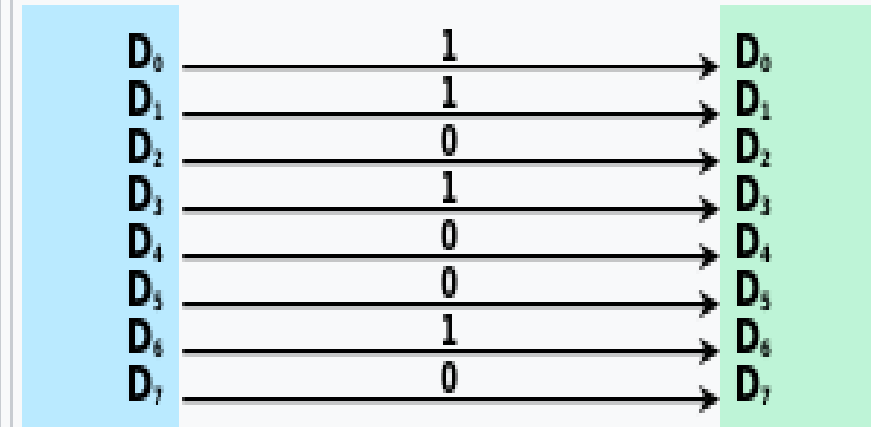
Types of Data Transmission



Parallel interface example

Transmitter (TX)

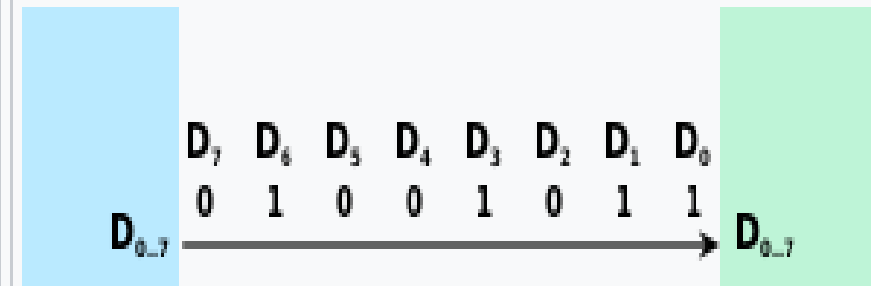
Receiver (RX)



Serial interface example

Transmitter (TX)

Receiver (RX)



MSB AND LSB

Data Transmission (Terminology)

Comparison between Serial and Parallel transmission

Sr. No.	Factor	Serial	Parallel
1.	Number of bits transmitted at one clock pulse	One bit	n bits
2.	No. of lines required to transmit n bits	One line	n lines
3.	Speed of data transfer	Slow	Fast
4.	Cost of transmission	Low as one line is required	Higher as n lines are required.
5.	Application	Long distance communication between two computers	Short distance communication. like computer to printer.

Data Transmission (Terminology)

Synchronous Data Transfer

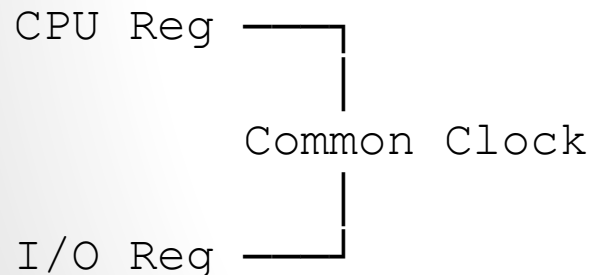
S (Sender) R (Receiver)

→ **Common clock**

- Two units share a **common clock**
- Data transfer happens according to this shared clock signal

Example:

- CPU register ↔ I/O register
- Both are driven by the **same clock**



- Timing is fixed
- Easy to design

Asynchronous Data Transfer

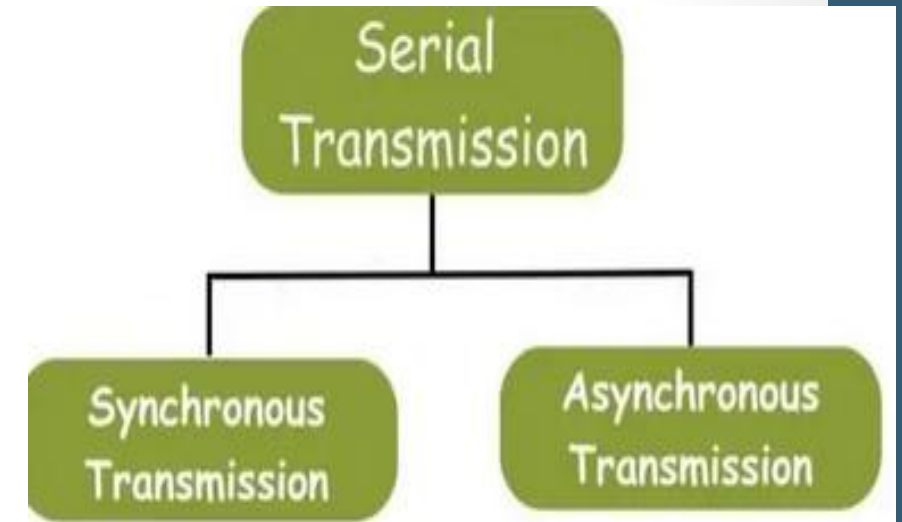
S (Sender) R (Receiver)

→ **Private / separate clocks**

- Two units are **independent**
- Each unit has its **own clock**
- Both devices must **coordinate** to transfer data

Example:

- CPU register (fast clock)
- I/O register (slow clock)
- Speed may vary (CPU fast, I/O slow)



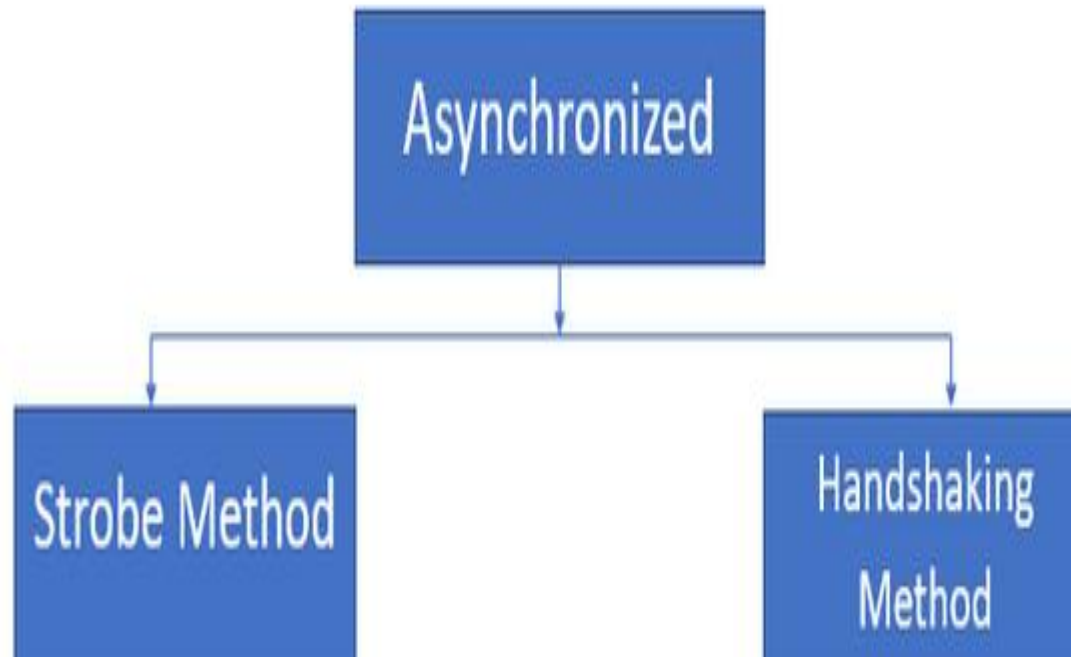
Asynchronous Data Transfer

Where You can use Asynchronous Data transfer?

- When speed changes significantly (CPU fast, I/O slow), we use **asynchronous data transfer**
- **Requires Control Signals** to be transmitted between Sender and receiver to **indicates** which time data is transfer and receive.

Asynchronous Data Transfer

Asynchronous Data Transfer Methods :-

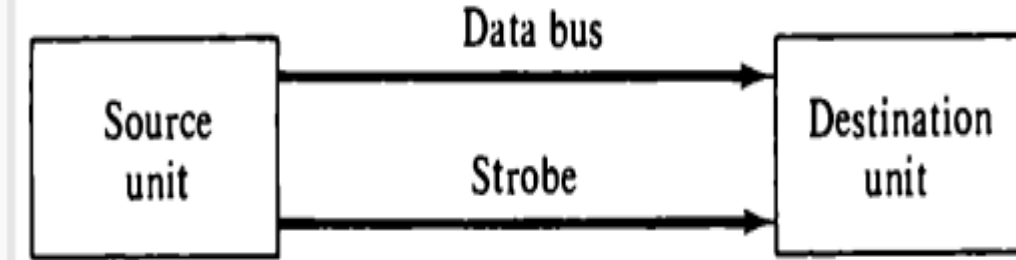


Asynchronous Data Transfer with Strobe Control

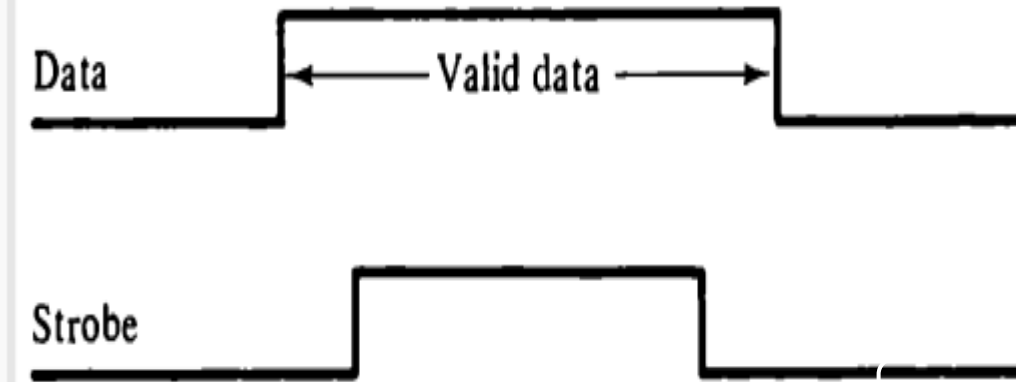
❖ Strobe may be activated by either source and destination unit.

Source-initiated strobe for data transfer :

- The **data bus carries the binary information** from source unit to the destination unit.
- The **strobe is a single line** that informs the destination unit when a valid data is available in the bus.
- The source unit **first places the data** on the data bus.
- **After a delay** to ensure that the data settle to a bus, the source activates the **strobe pulse**.
- The information on the data bus and the strobe signal remain in the active state for a sufficient time period to allow the destination unit to receive the data.
- **After some period of time strobe pulse disable and then after data bus is also clear.**
- **Strobe plus disable it means data bus has no available valid data to transfer.**
- **Example – (typeing msg & send after internet start reached to destination signal so,msg – data bus and intternet – strob)**



(a) Block diagram



(b) Timing diagram

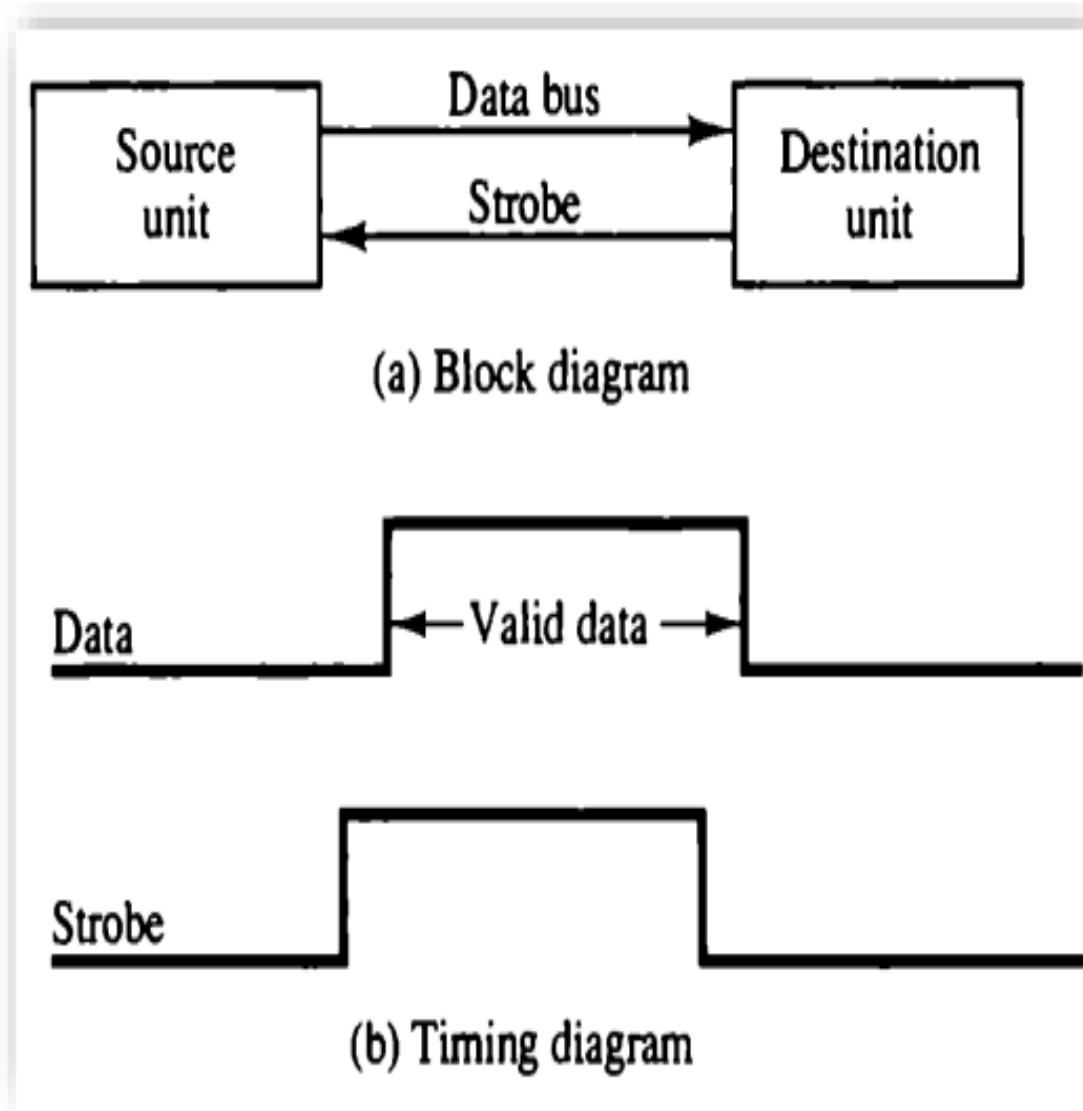
Asynchronous Data Transfer with Strobe Control

Destination-initiated strobe for data transfer:

- Here the **destination unit activates** the strobe pulse, informing the source to provide the data.
- The source unit responds by placing the requested **binary information on the data bus**.
- The data must be valid and remain in the bus long enough for the destination unit to accept it.
- The destination unit then disables the strobe. The source removes the data from the bus after a time interval.

(Example – before send msg I will check online , then after I put msg.

Strobe – online and data bus – msg)



Asynchronous Data Transfer

- The disadvantage of the strobe method is that **the source unit that initiates the transfer has no way of knowing whether the destination unit has actually received the data item that was placed in the bus.**
- Similarly, a **destination unit that initiates the transfer has no way of knowing whether the source unit has actually placed the data on the bus**

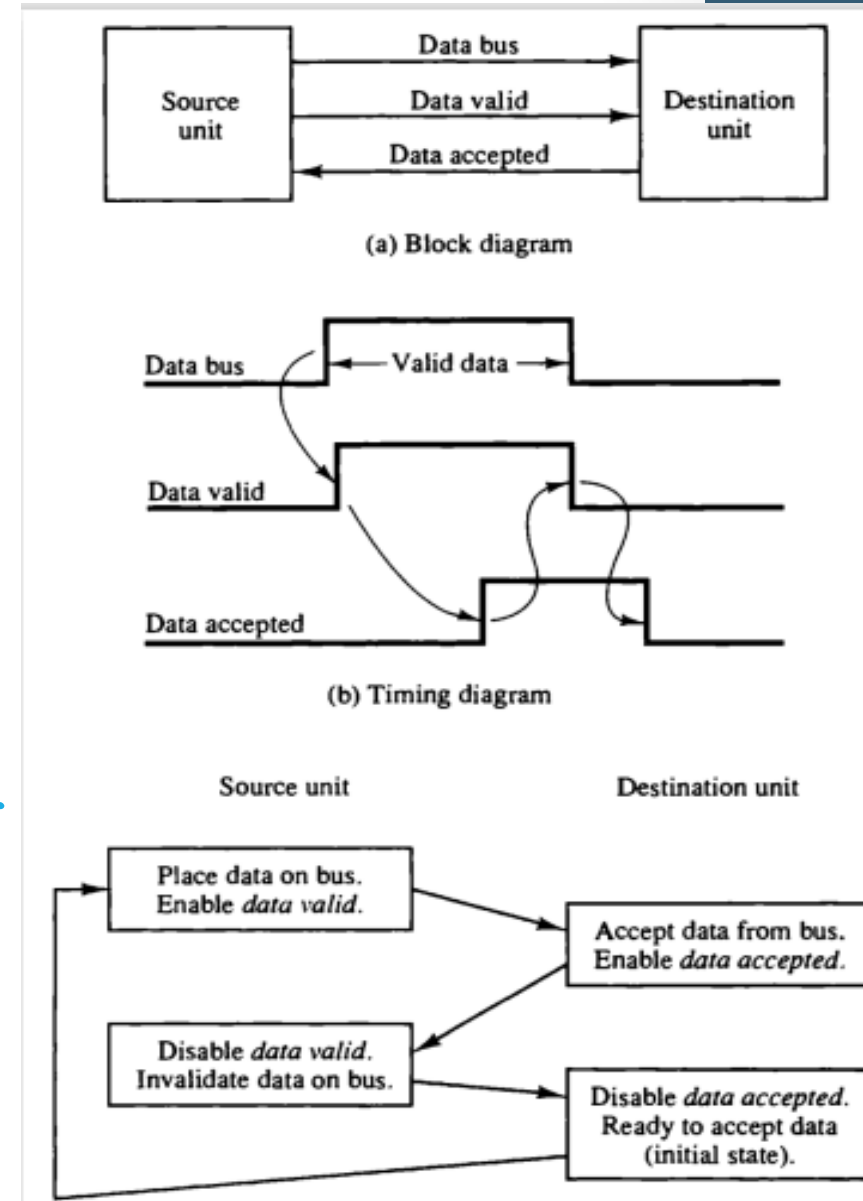
The handshake method solves the problem of Strobe method by introducing a **second control signal that provides a reply to the unit that initiates the transfer.**

Strobe Control signal + ACK signal = Handshaking Method

Asynchronous Data Transfer with Handshaking method

Source-initiated transfer using handshaking :-

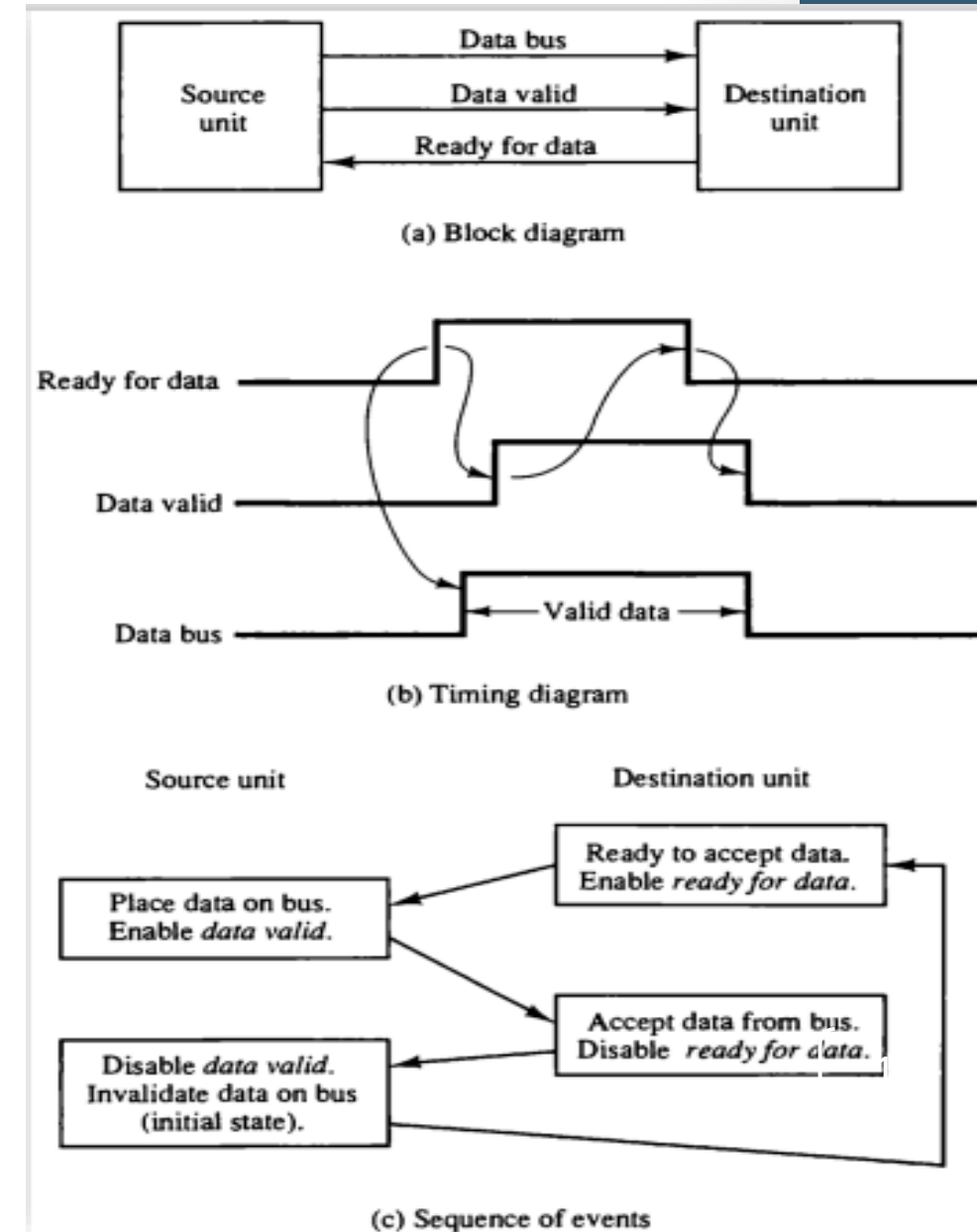
- One control signal is in the same direction as source to the destination.
- It is used by the source unit to **inform** the destination unit whether there are **valid data in the bus**.
- The other control line is in the other direction from the destination to the source & is used by the destination unit **to inform the source, it can accept data**.
- The two handshaking lines the data valid, which is generated by the source unit, and data accepted, generated by the destination unit, the **timing diagram shows the exchange of signals between the two units**.



Asynchronous Data Transfer with Handshaking method

Destination-initiated transfer using handshaking:

- The only difference between the source-initiated and the destination-initiated transfer is in **their choice of initial state**.
- The source unit in this case does not place data on the bus **until it receives the ready for data signal from the destination unit**.
- From there on, the handshaking procedure follows the same pattern as in the source-initiated case.



Modes of transfer

- Binary information received from an external device is usually **stored in memory** for later processing.
- Data transfer may be handled three possible modes:
 1. Programmed I/O
 2. Interrupt-initiated I/O
 3. Direct memory access (DMA)

Modes of transfer

1) Programmed I/O

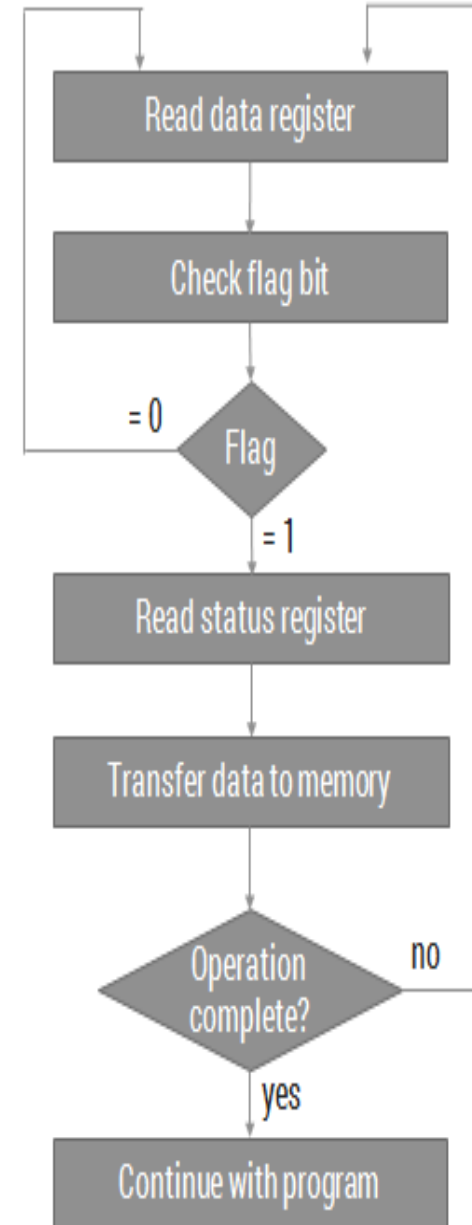
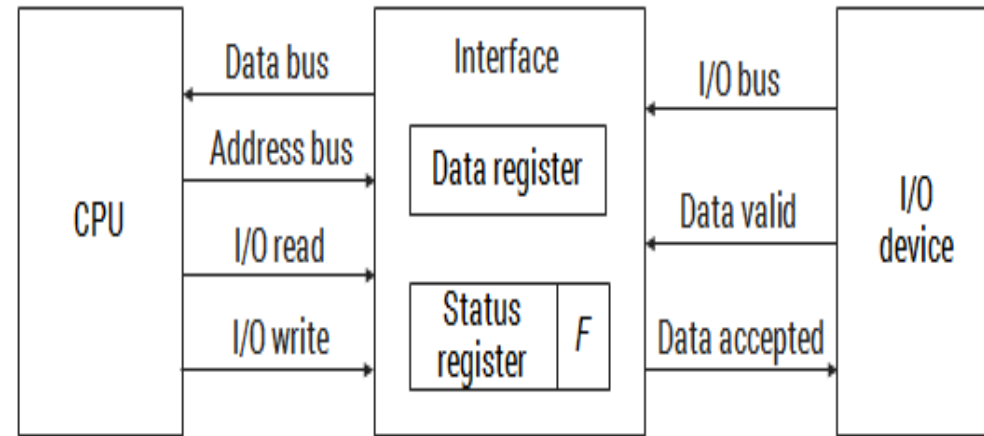
- In the programmed I/O method, the I/O device **does not have direct access** to memory.
- When CPU Initiates transfer the data to I/O devices, it first check the I/O is free or not, after that when I/O is free then transfer the original data and if it is busy then CPU going to waiting mode.
- It is very time consuming Process because CPU is in waiting stage.
- In programmed I/O, the CPU stays in the program loop until the I/O unit indicates that it is ready for data transfer.

Modes of transfer

How to check CPU ,I/O is free? (refer slide- 5)

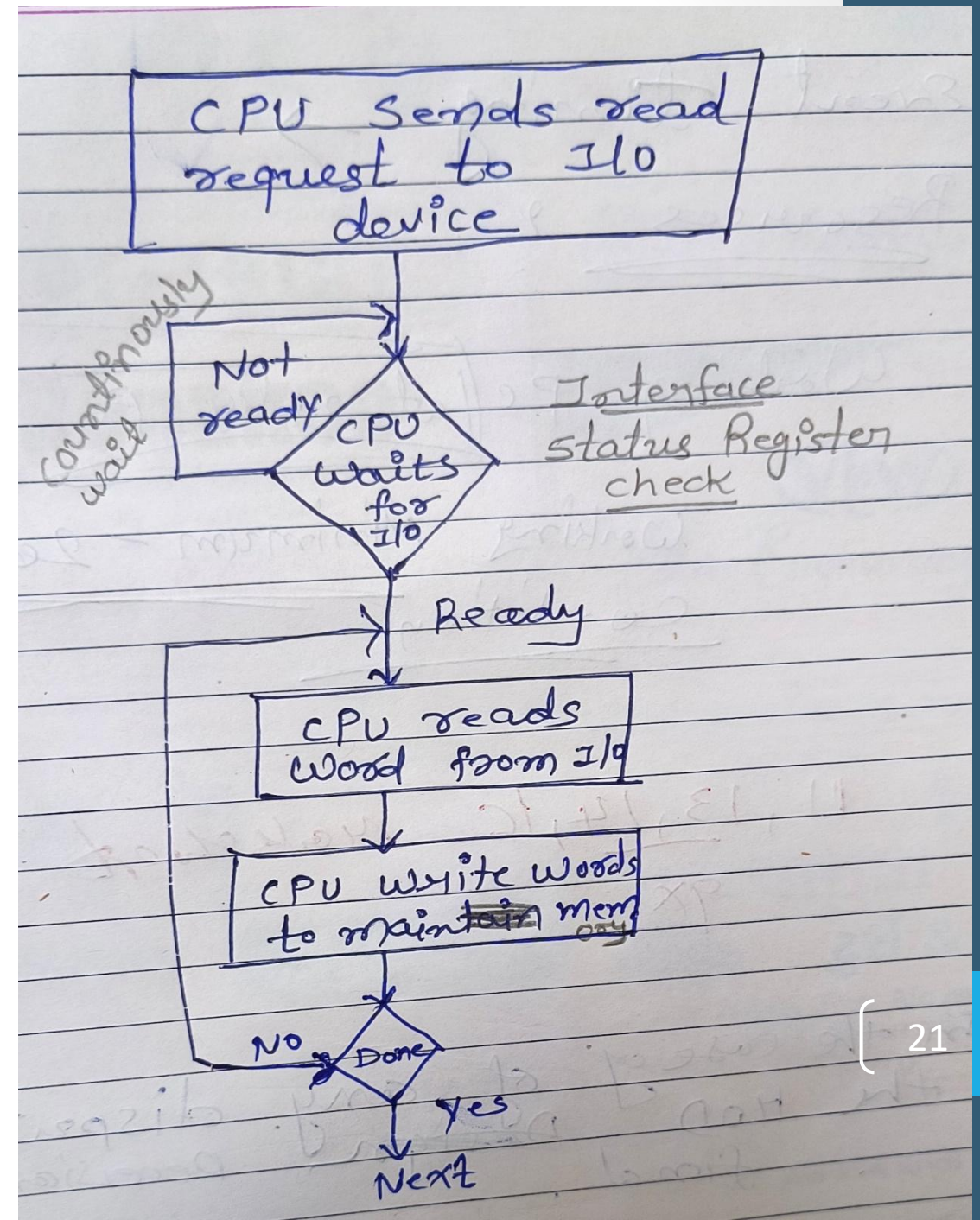
I/O Interface the status bit register is enable , it means I/O is free and ready to data transfer.

Programmed I/O



Modes of transfer

- Example of Programmed I/O for input output data transfer.
- In Programmed I/o , CPU makes a request and then CPU stays in Program loop (polling) until the I/O devices indicates that it is ready for data transfer.
- CPU reads word from device and after that CPU write word to main memory.
- It is very time consuming Process because it keep CPU busy needlessly.



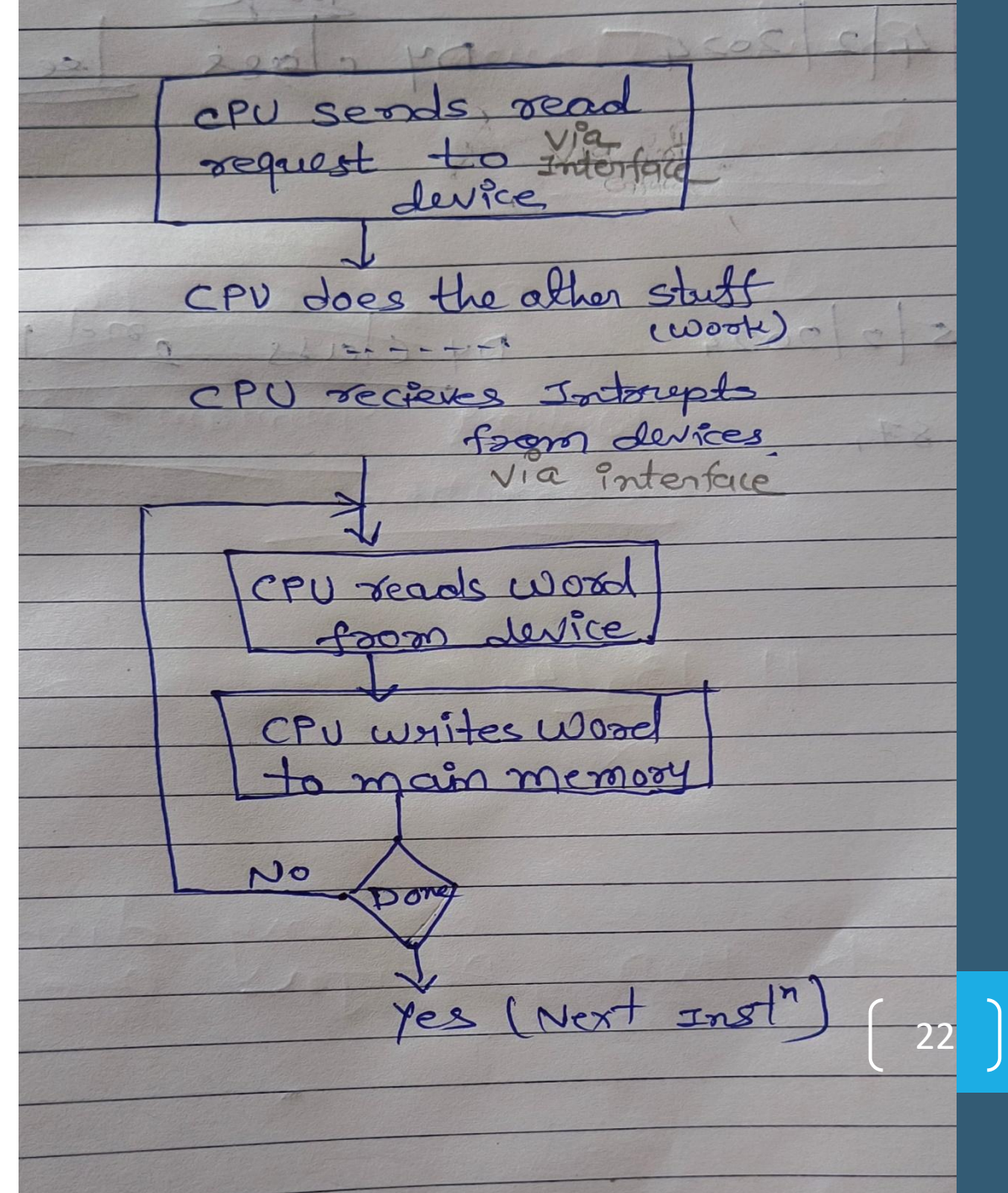
Modes of transfer

2) Interrupted Initiated I/O

What is Interrupt ?

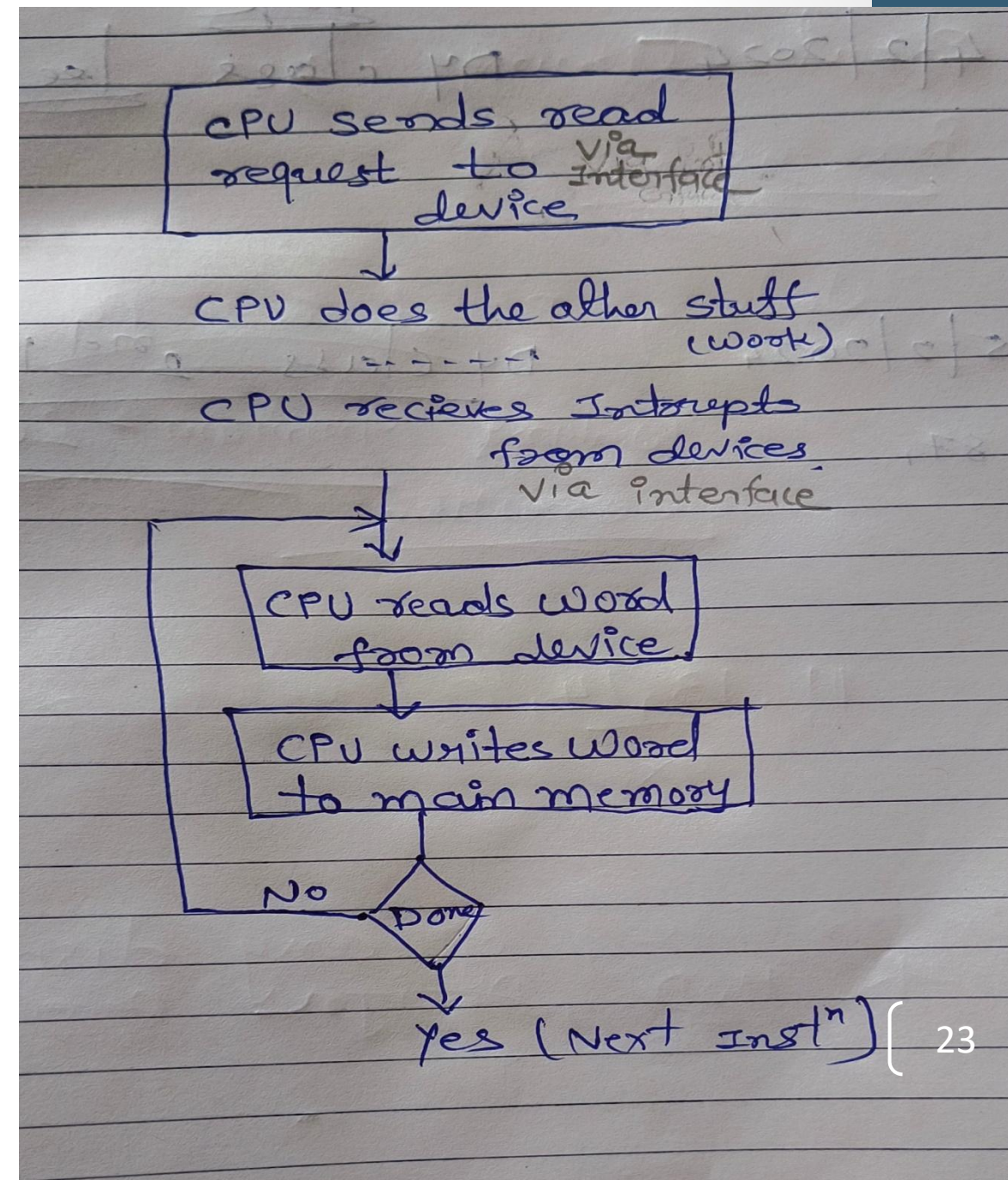
Interrupt is a high Priority signal which is generated by either h/w device and s/w Program to get the immediate action of the CPU.

- Instead of continuous monitoring of CPU, interface will be informed to interrupted request signal, when data are available from device(I/O).
- Meanwhile, CPU Produce to execute another Program and interface keeps monitoring the device(I/O).



Modes of transfer

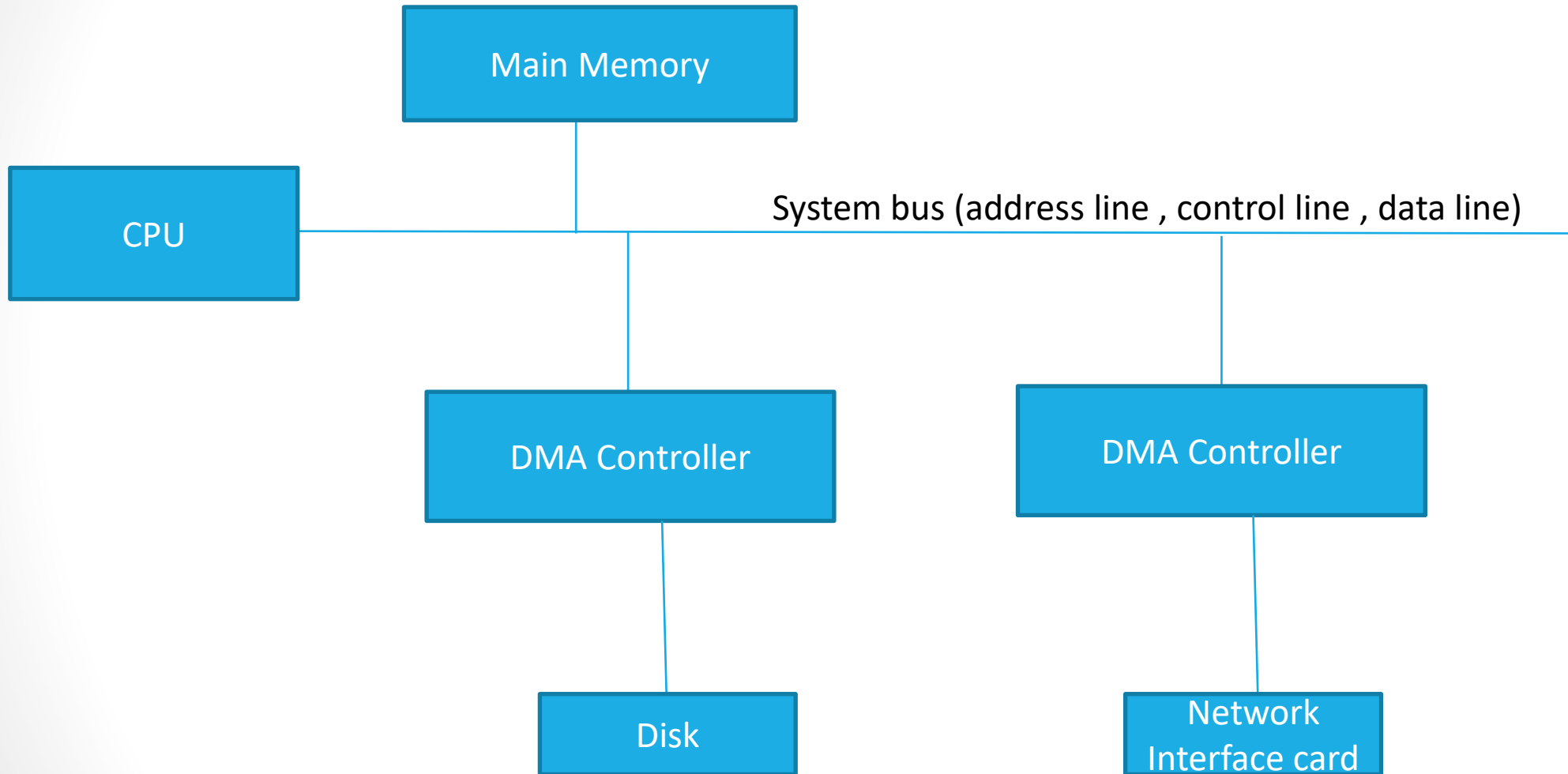
- When device is ready for data transfer, it generates interrupt request to interface.
- After Detecting the external interrupt signal, CPU stops the task and performing I/O data transfer Process.



DMA

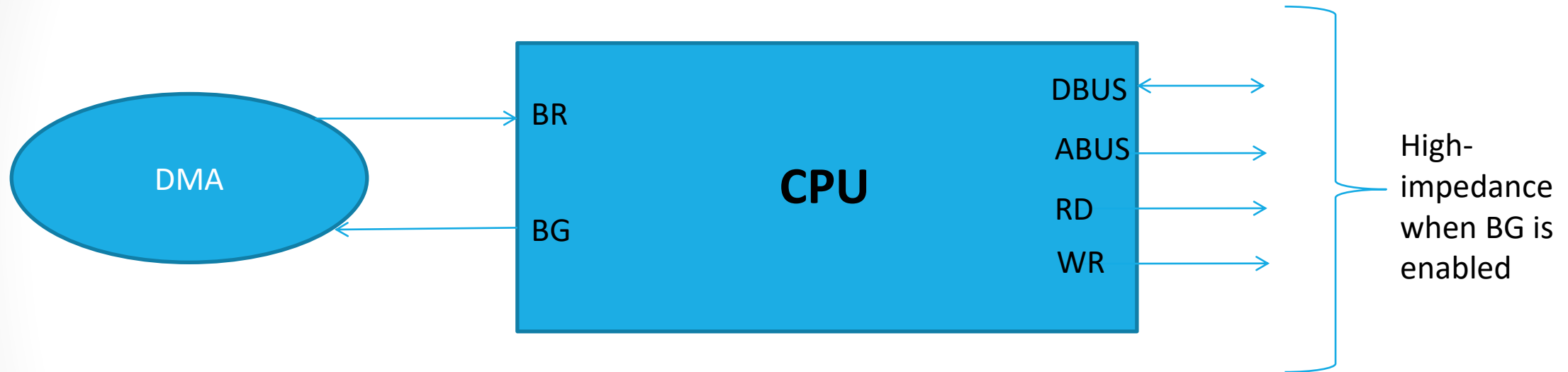
- DMA is a I/O technique that **Provides direct access to the main memory while CPU is Temporally disabled,**
- Previously without DMA data transfer using I/O to CPU and CPU to main memory, that meaning time consuming.
- The Process is managed by **chips known as “DMA Controller”.**
- In DMA, both CPU and DMA controller have access to main memory via a system bus.
- **During DMA transfer the CPU is idle and has no control of system bus.**
- A “DMA controller ” temporarily borrows the address bus, data bus and control bus from the CPU and transfers data between I/O devices and main memory.

DMA



DMA

How to make(reached) CPU in idle-State ?

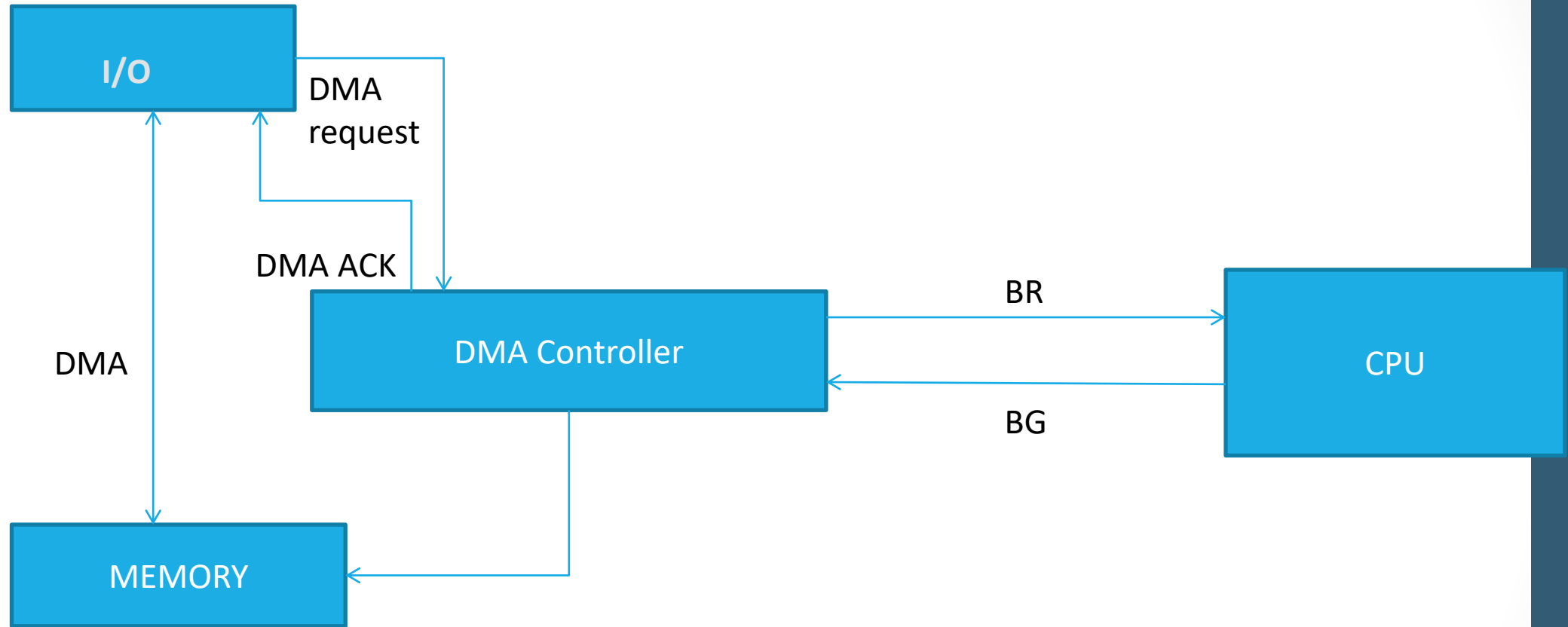


High-impedance state means it disconnected to CPU.

It has 2 special Control signals

- 1) Bus request (BR) signal
- 2) Bus Grant (BG) signal

DMA

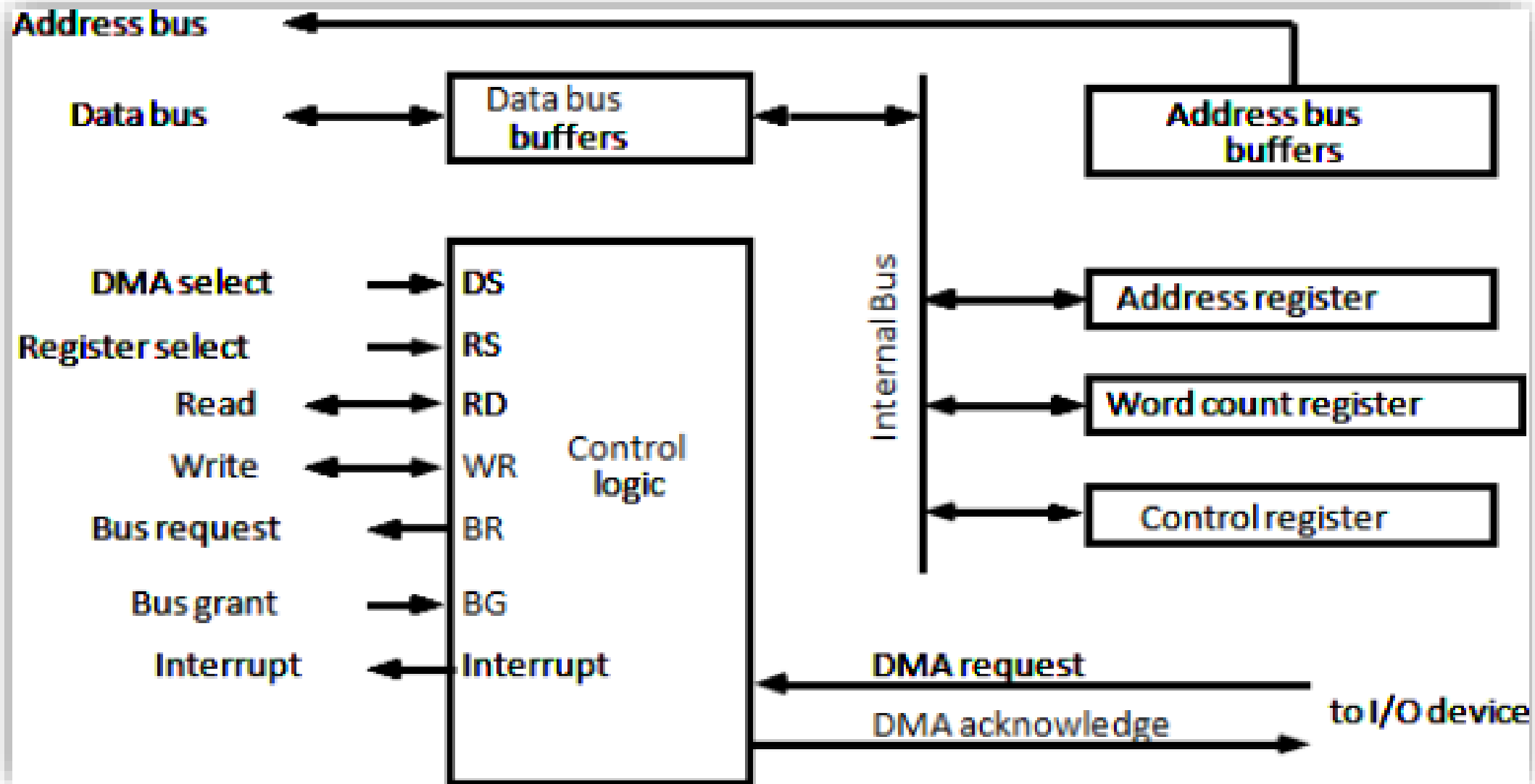


DMA

DMA working :-

- 1) I/O decided to perform data transfer then DMA controller send request.
- 2) DMA controller send BR signal to CPU Then CPU address, data, read and write signals to disable then BG signal enable.
- 3) CPU activated BG signals as a output and then CPU is on ideal state.
- 4) Now, all the control have DMA.
- 5) DMA also ACK to I/O devices to known all control is accessed directly.
- 6) DMA transfer the request to main memory to transfer data(Step 1).
- 7) When operation Performed then DMA Terminated to send BUS request line to CPU, CPU send BG signal take back all control.

DMAC (Block diagram of DMA Controller)



DMAC (Block diagram of DMA Controller)

- **DMA controller - Interface which allows I/O transfer directly between Memory and Device, freeing CPU for other tasks.**
- The registers in the DMA are selected by the CPU (Communication) to enabling the DS (DMA select) and RS (register select) inputs.
- The RD (read) and WR (write) inputs are bidirectional. At a time either one is activated.
- When the BG (bus grant) input is 0, the CPU can communicate with the DMA registers through the data bus to read or write operation.
- When BG= 1, the CPU give up the buses and the DMA can communicate directly with the memory.
- The DMA communicates with the external peripheral(I/o devices) through the request and acknowledge lines.

DMAC (Block diagram of DMA Controller)

- The DMA controller has three registers: an **address register**, a **word count register**, and a **control register**.
- **Address register hold the address which is operation perform by the memory**
- The address register contains an address to specify the desired location in memory, the address register is incremented after each word transfer to memory.
- The word count register holds the number of words to be transferred And also decremented by one after each word.
- The control register specifies the mode of transfer.

Priority Interrupt

CPU is connected with more than one device and all devices are capable to send the signals, if all devices send interrupts at a same time then CPU which device give to service first?



Multiple devices send simultaneously interrupt to CPU then first CPU finds, which of interrupt is higher priority interrupt?

CPU must also decide Which device to service first? Is called Priority interrupt system.

Priority interrupt system is used to determine which device to give the services first.

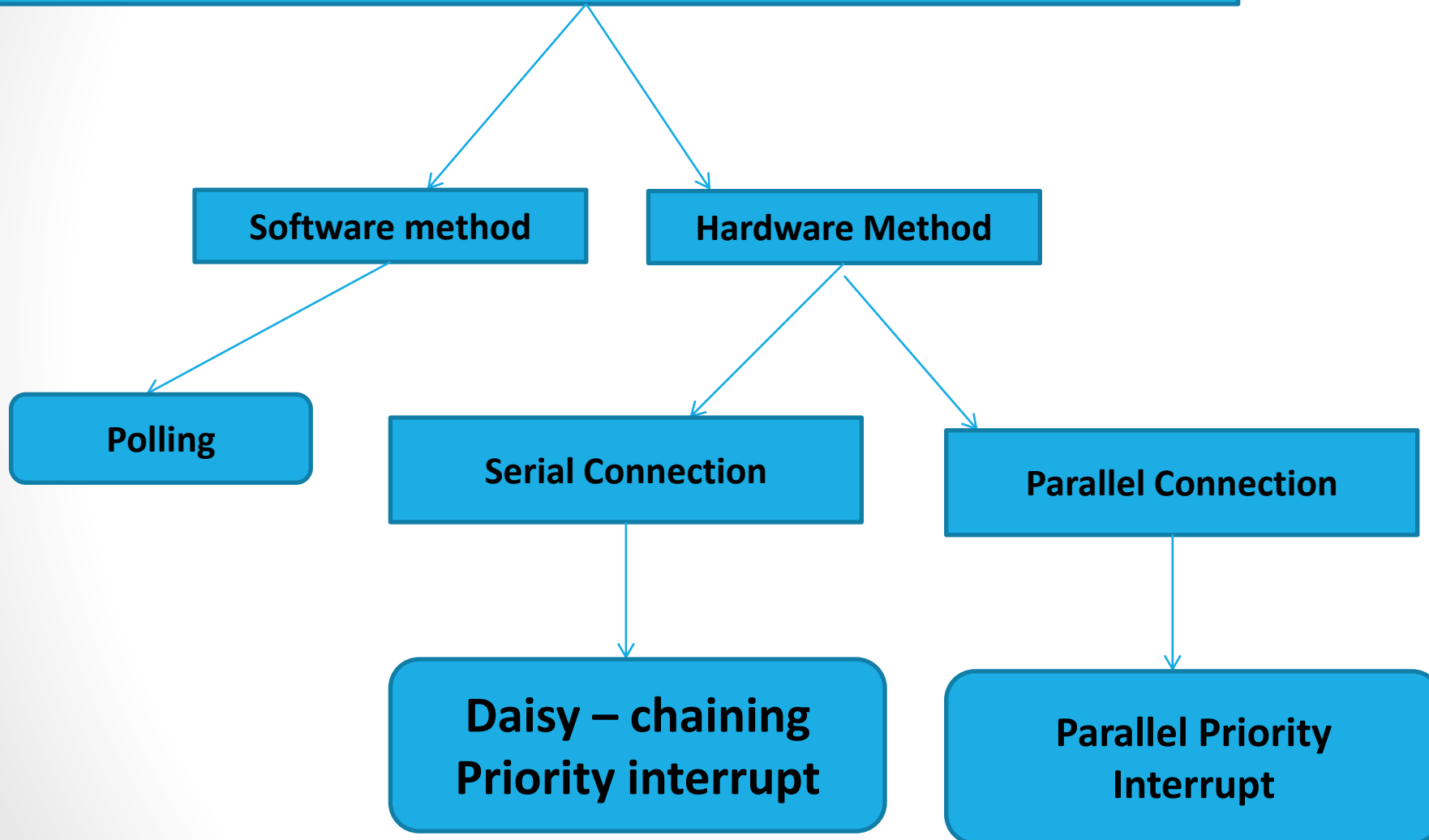
Example -

Devices with high speed transfers such as magnetic disks are given high priority, and slow devices such as keyboards receive low priority. When two devices interrupt the computer at the same time, the computer services the device, with the higher priority first.

Device of high speed transfer are given higher Priority.

Priority Interrupt

Establishing the priority of simultaneous interrupts



Priority Interrupt

- Priority interrupts by software (Polling)
- It is a software method of establishing priority of simultaneously interrupt.
- CPU continuously to poll the device and asked whether you have generated interrupted or not?
- When CPU detects interrupt then CPU generate branches using common ISR,CPU Gives service with higher Priority device,(Interrupt perform specified task and that task is defined special function and it is called ISR) AFTER handling interrupt cpu back to done original task.
- And also CPU start Poll to higher priority to lower Priority.
- Problem is that it is time consuming Process so the another method is used Hardware.

Higher to lower asked all device for interrupt .



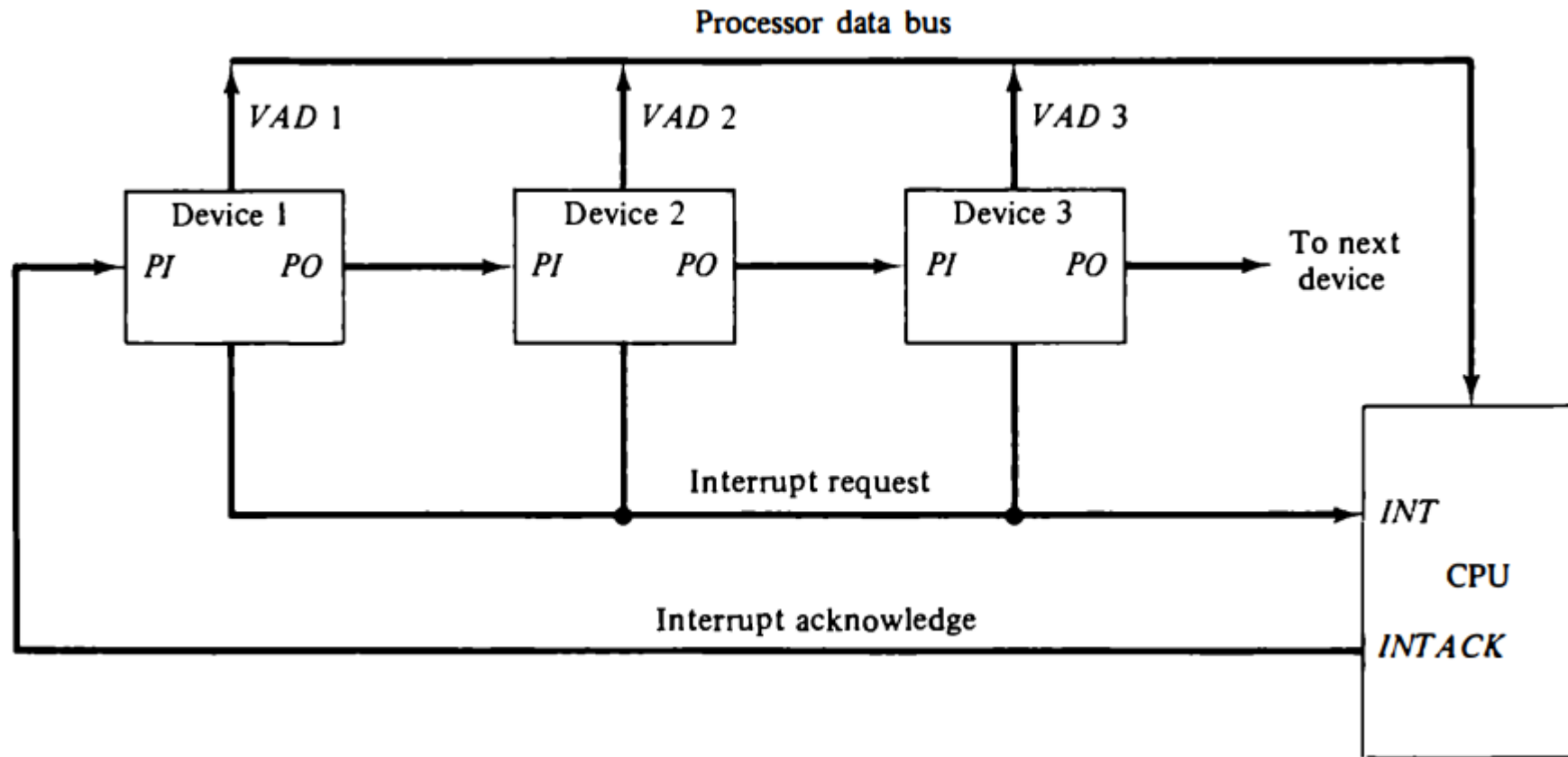
Priority Interrupt

- Priority interrupts by Hardware.(all worked done by Priority manager)

Priority manager which accepts all the interrupt request to determine the highest Priority request.(Hardware Priority interrupt unit)

No need polling because all the decision are established by Hardware Priority interrupt unit.

Priority Interrupt - Daisy-Chaining Priority



Interrupt generated
PI = 1 , PO = 0

NOT Generated
PI = 1 , PO = 1

INTACK Block
PI = 0, PO = 0

- It contains of serial connection of all devices that request a interrupt.
- Device of higher Priority is placed first.
- Device that wants attention send the interrupt request to the CPU (through the common interrupt request (INT) line CPU then sends the interrupt acknowledge signal (INTACK).

Priority Interrupt - Daisy-Chaining Priority

- If device had requested the interrupt, it places VAD (Vector address) on the the bus, and it blocks the signal by placing 0 in P0 (Priority OUT) i.e. $[PI = 1 \ \& \ P0 = 0]$
- If device had not requested the interrupts, it pass the ack signal(INTACK) to next device through P0 placing by 1 i.e. $[PI = 1 \ \& \ P0 = 1]$ this Process continuous until appropriate device is found.
- IF device gets 0 in its PI input, it generates 0 at the P0 output to tell other devices that acknowledgment signal [INTACK] has been blocked (occupied).
- Vector address is a address of ISR (Interrupt service routing).
- The device whose $PI = 1 \ \& \ P0 = 0$ is the device with higher Priority that is requesting for interrupts and this device place its VAD on data bus.

Input – Output Processor(IOP)

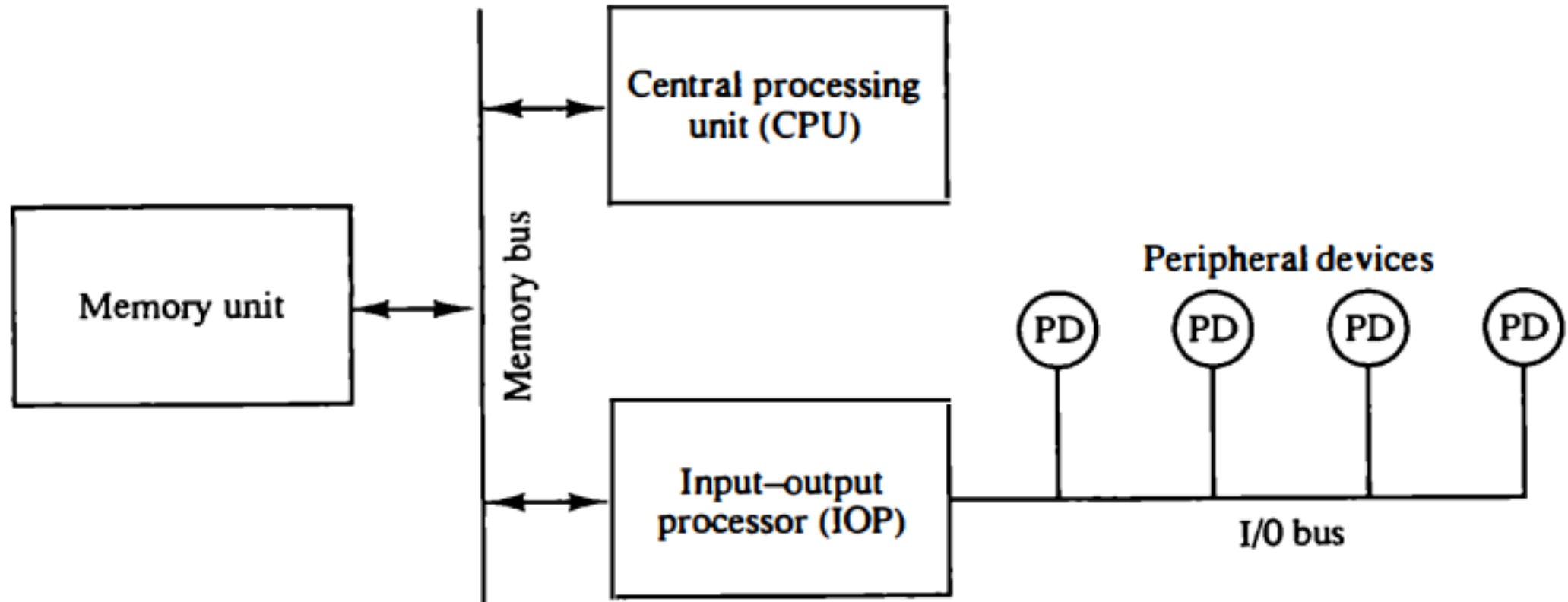


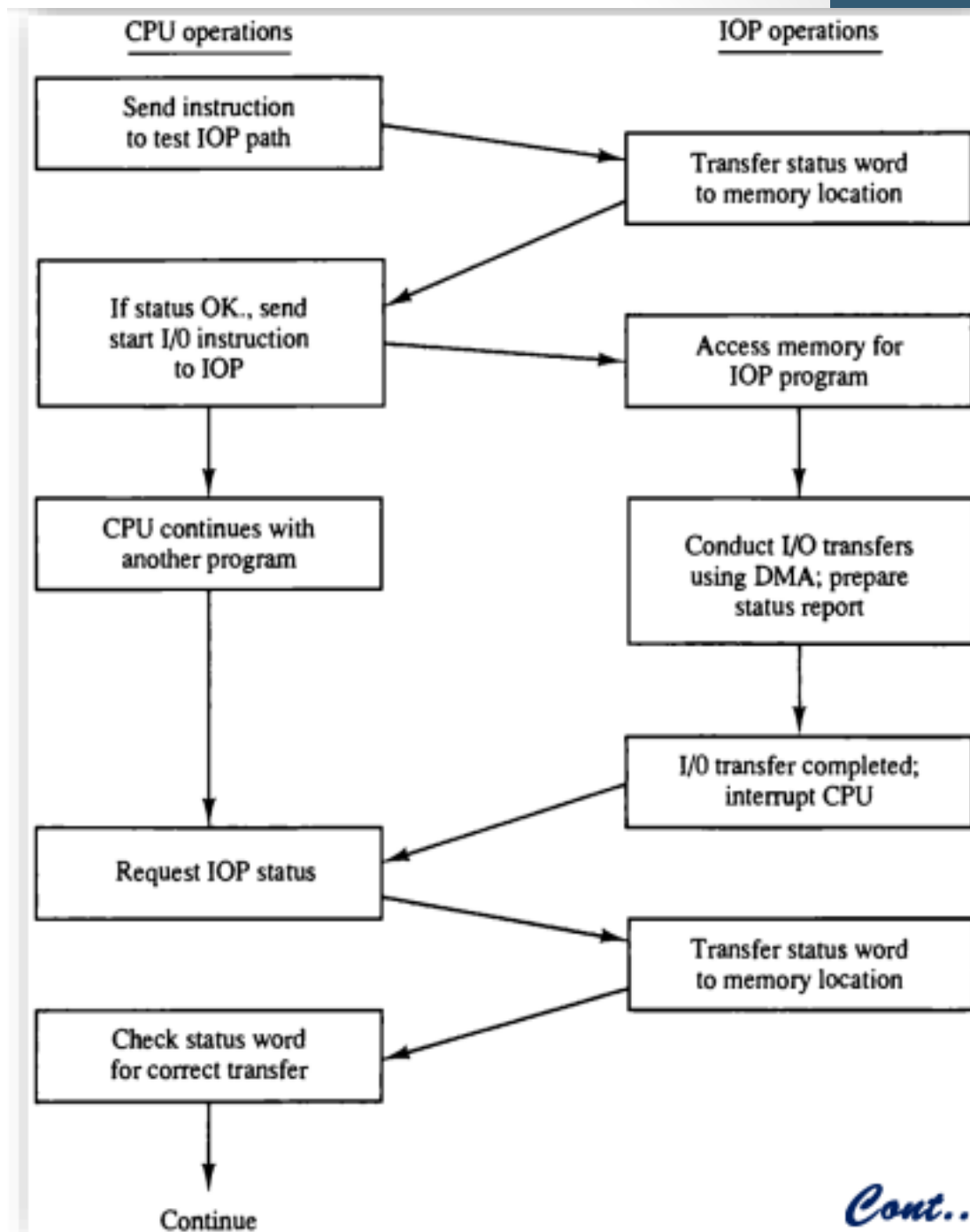
Figure 11-19 Block diagram of a computer with I/O processor.

Input – Output Processor(IOP)

- IOP is similar to a CPU except that it is **designed to handle the details of I/O** processing.
- Unlike the DMA controller that must be setup entirely by the CPU, the IOP can fetch and execute its own instruction.
- IOP are specifically designed to facilitate I/O transfers.
- The IOP provides a path for transfer of data between various peripheral devices and memory unit.
- It control and managed the I/O Tasks, and fetch and executes the instruction from i/o devices.
- When IOP perform task to the main memory at that time if CPU Deal with main memory so it releases all the control to memory for some period of time called **cycle stealing**.

CPU IOP Communication

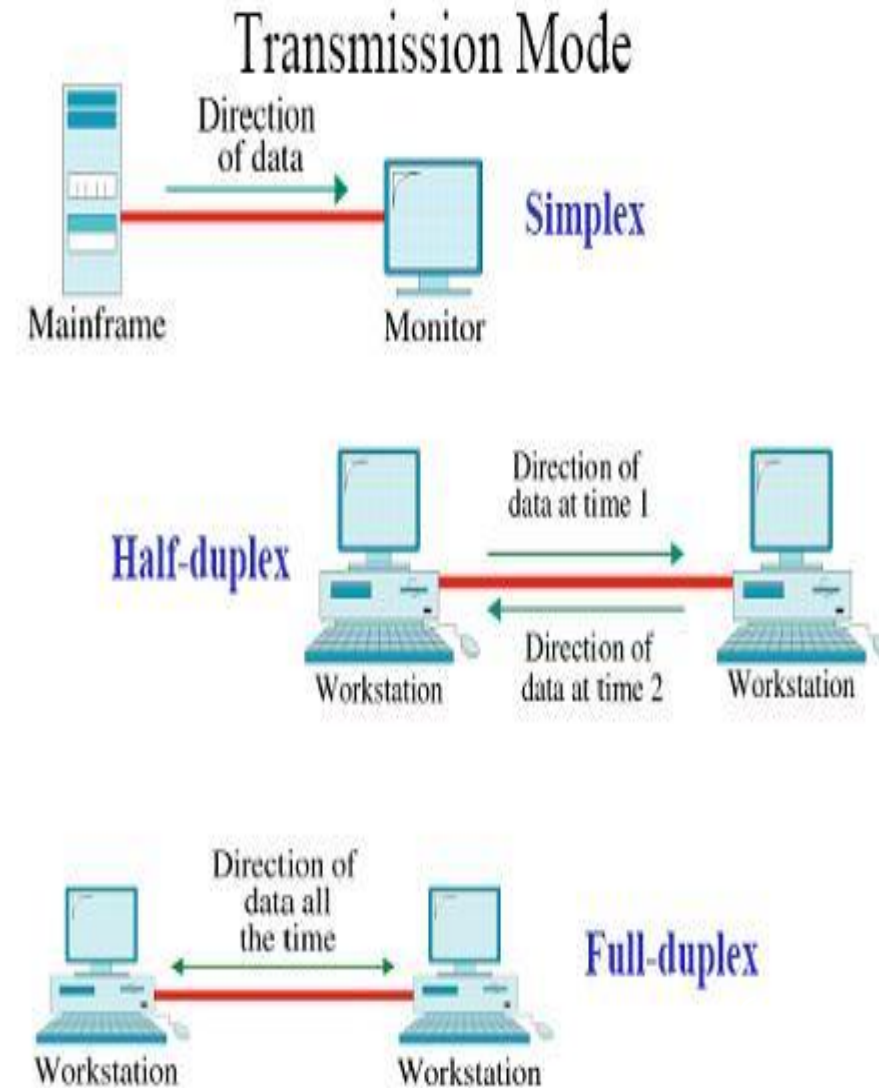
- 1) CPU send request and test I/O path is clear or not send the data .
- 2) I/O check in status word device is ready , error free , overflow.
- 3) If ok then CPU tell IOP to start I/O instruction.
- 4) I/O processor to access memory meanwhile CPU can do their other stuff
- 5) I/O Transfer completed and then interrupt CPU
- 6) CPU request to I/O to check status
- 7) Transfer status word to memory location.
- 8) Check status word for correct transfer.



Cont...

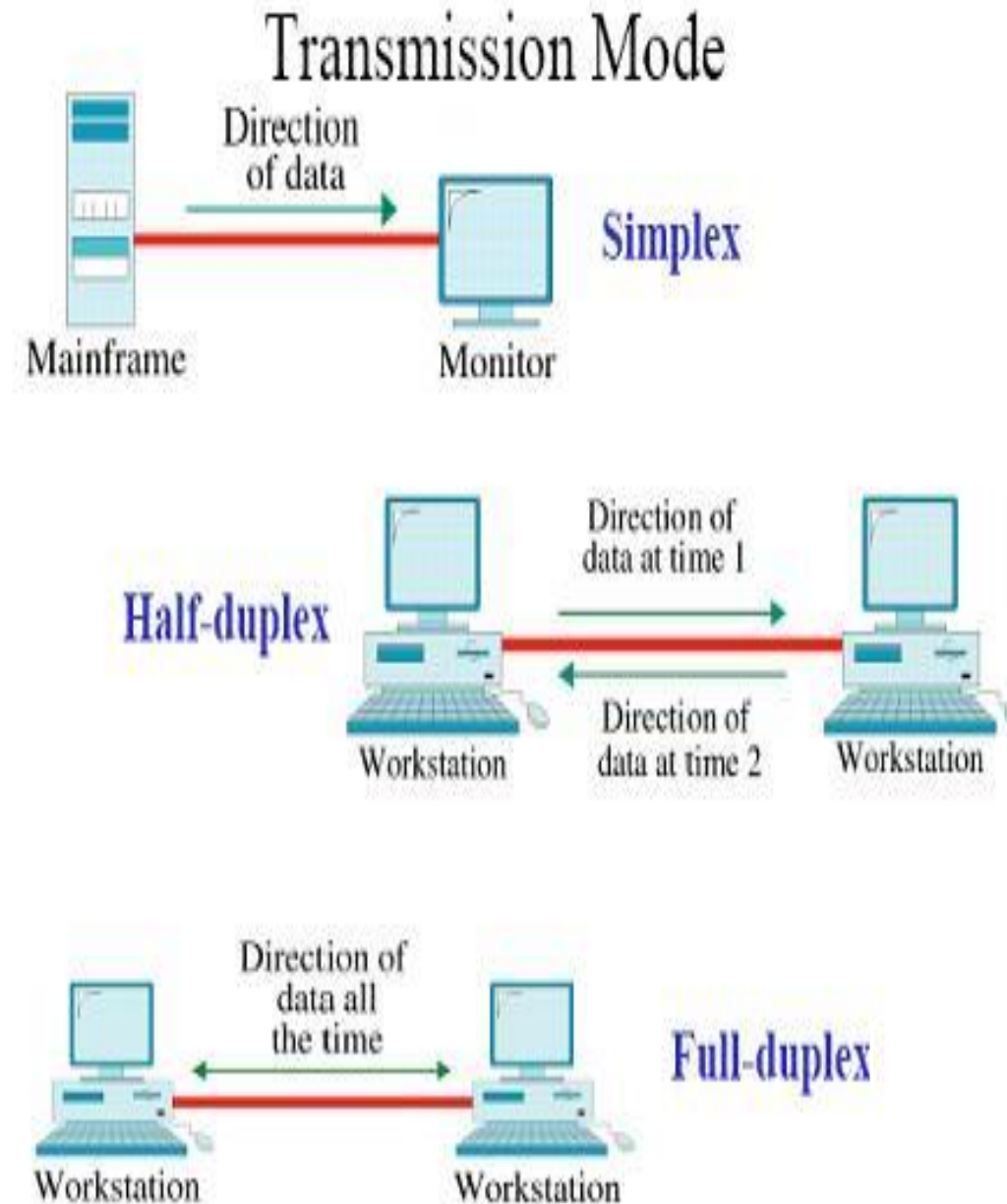
Serial Communication

- Serial communication takes place in many ways depending on type of transmission modes and data transfer.
- Data can be transmitted between *two points* in **three different modes**: simplex, half-duplex, or full-duplex.
- A simplex line carries information in one direction only. This mode is rarely used in data communication because the receiver cannot communicate with the transmitter.
- Examples : radio and television broadcasting.



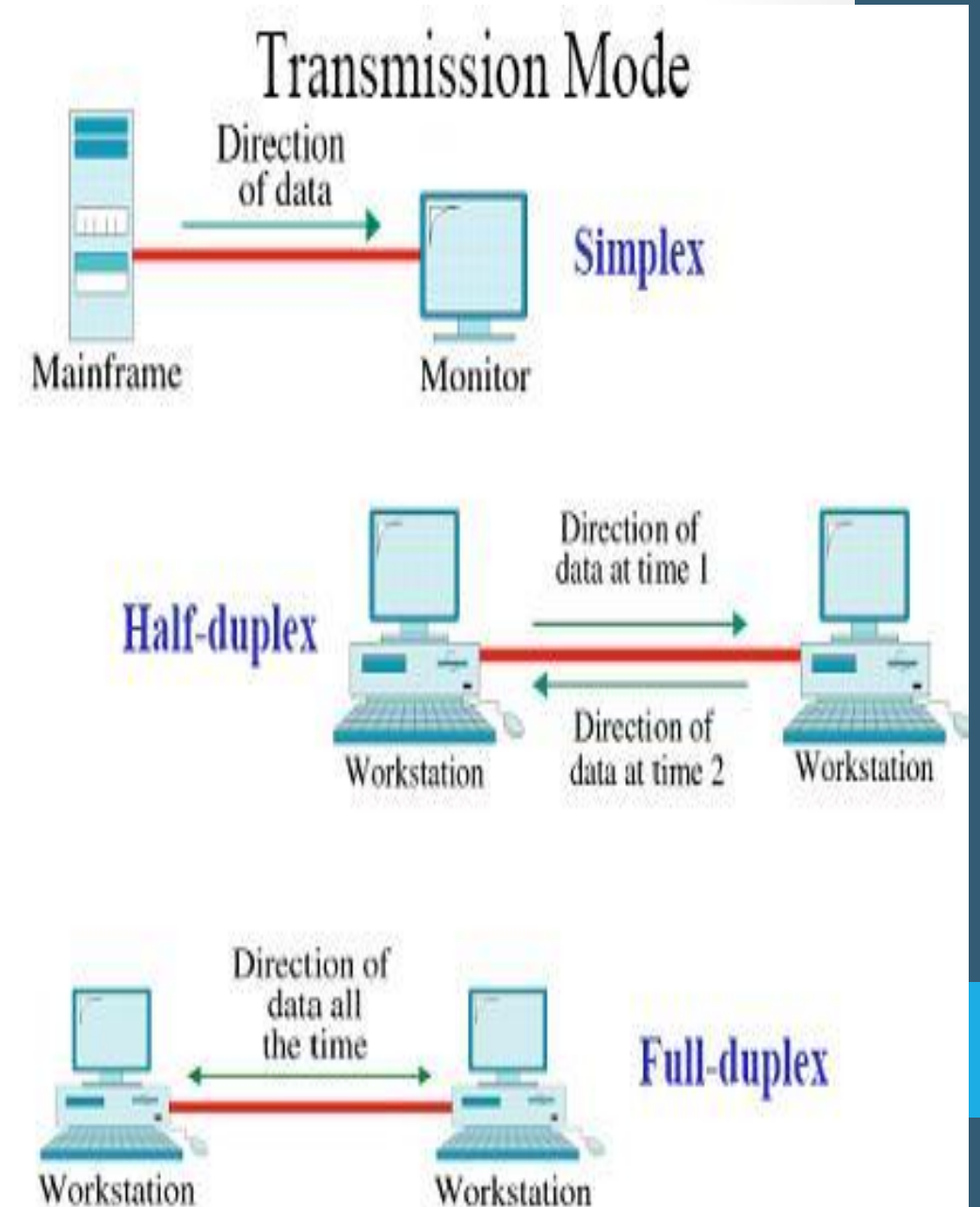
Serial Communication

- A **half-duplex** transmission system is one that is capable of transmitting in both directions but data can be transmitted in **only one direction at a time**. A pair of wires is needed for this mode. A common situation is for one modem to act as the transmitter and the other as the receiver.
- Example: Walkie-Talkie



Serial Communication

- A full-duplex transmission can send and receive data in both directions simultaneously. This can be achieved by means of a four-wire link.
- Example: Audio Calling



QUESTIONS ASKED IN GTU EXAM

- **Q.1 Elaborate CPU-IOP communication.**
- **Q.2 Explain DMA in brief.**
- **Q.3 Prepare flowchart of CPU-IOP communication.**
- **Q.4 Write a note on asynchronous data transfer.**
- **Q.5 Which are the different ways to transfer data to and from peripheral devices? Explain any one of them in detail.**
- **Q.6 Elaborate CPU-IOP communication.**
- **Q.7 Briefly explain DMA.**
- **Q.8 Discuss source-initiated transfer using handshaking in asynchronous data transfer.**
- **Q.9 Write a note on asynchronous data transfer.**
- **Q.10 Explain the working of Direct Memory Access (DMA).**

THANK YOU